

Schema.graphqls

Кореневий тип для читання даних

```
type Query {  
  # Поле з аргументом  
  getTripById(id: ID!): Trip  
  getAllTrips: [Trip]  
}
```

Кореневий тип для зміни даних

```
type Mutation {  
  # Поле з аргументом, один з яких InputObjectType  
  addParticipantToTrip(tripId: ID!, newParticipant: ParticipantInput!): Trip  
}
```

Вхідний тип даних для мутації

```
input ParticipantInput {  
  id: ID!  
  name: String!  
  role: String!  
}
```

Типи даних (3 рівні вкладеності)

Рівень 1: Подорож

```
type Trip {  
  id: ID!  
  destination: String!  
  date: String!  
  participants: [Participant] # Перехід на 2 рівень  
}
```

Рівень 2: Учасник

```
type Participant {  
  id: ID!  
  name: String!  
  role: String!  
  preferences: Preference # Перехід на 3 рівень  
}
```

Рівень 3: Вподобання учасника

```
type Preference {  
  budget: Int  
  preferredTransport: String  
}
```

type Query { — Це спеціальний, зарезервований тип у GraphQL. Усе, що написано всередині нього, відповідає **виключно за читання** даних. Якщо фронтенд хоче щось дістати, він звертається сюди.

getTripById(id: ID!): Trip — Оголошуємо метод. Він приймає аргумент id типу ID (і цей id є обов'язковим, бо стоїть !). Після двокрапки вказано, що метод поверне: рівно один об'єкт нашого типу Trip.

getAllTrips: [Trip] — Метод без аргументів. Повертає масив (список) об'єктів Trip.

type Mutation { — Ще один спеціальний тип. Тут лежать усі методи, які **змінюють стан сервера** (додають, видаляють, оновлюють дані).

addParticipantToTrip(...) — Метод, який приймає два обов'язкові аргументи:

tripId: ID! — ID поїздки, куди ми додаємо людину.

newParticipant: ParticipantInput! — самі дані нової людини.

: **Trip** в кінці означає, що після успішного додавання сервер поверне нам оновлену поїздку, щоб ми одразу побачили зміни.

input (а не type) — Це дуже важливий момент! У GraphQL **не можна** передавати звичайні type (вихідні об'єкти) як аргументи в мутації. Для передачі складних об'єктів з клієнта на сервер обов'язково треба створювати спеціальний тип input. Це вимагав **пункт 4** вашого завдання.

Усередині ми просто перелічуємо, що для створення учасника нам обов'язково треба передати його id, name та role.

Описуємо, як виглядає сутність "Подорож". Вона має обов'язкові поля id, пункт призначення та дату.

participants: [Participant] — Ось тут відбувається магія зв'язку (перехід на 2-й рівень). Ми кажемо, що поїздка містить список учасників. Зверніть увагу, тут **немає знака оклику**, бо при створенні нової поїздки список учасників може бути порожнім.

· **Класичне запитання викладача на захисті: "А чим відрізняється type від input?"**
Ваша ідеальна відповідь: "type використовується для того, щоб ВІДДАВАТИ дані клієнту (наприклад, як результат запити). А input використовується виключно для того, щоб ПРИЙМАТИ складні об'єкти від клієнта (наприклад, як аргумент у мутації)."

Fetchers

```
#public class GraphQLDataFetchers {  
  
    private final DataRepository repository;  
  
    // Конструктор приймає наш репозиторій з даними  
    public GraphQLDataFetchers(DataRepository repository) {  
        this.repository = repository;  
    }  
  
    #// Кастомний Fetcher 1: Отримання всіх поїздок  
    public DataFetcher<List<Trip>> getAllTripsDataFetcher() {  
        return dataFetchingEnvironment -> repository.getAllTrips();  
    }  
  
    #// Кастомний Fetcher 2: Отримання поїздки за ID (використовує аргумент)  
    public DataFetcher<Trip> getTripByIdDataFetcher() {  
        return dataFetchingEnvironment -> {  
            String tripId = dataFetchingEnvironment.getArgument("id");  
            return repository.getTripById(tripId);  
        };  
    }  
}
```

Рядок 1: Оголошуємо клас.

Рядок 2: Створюємо приватну змінну repository. Це посилання на наш клас DataRepository (де лежить розпарсений JSON з нашими поїздками)

Рядок 4-6: Це конструктор. Коли програма стартує (в Main), вона створює цей клас і передає йому готовий репозиторій з даними. Завдяки цьому наші "кухарі" (фетчери) мають доступ до "холодильника" (репозиторію).

Що це: Це метод, який повертає об'єкт типу DataFetcher. DataFetcher — це інтерфейс із бібліотеки GraphQL-Java, який має лише одне завдання: дістати дані для конкретного поля.

dataFetchingEnvironment -> ... — Це лямбда-вираз (коротка форма запису функції в Java). dataFetchingEnvironment (середовище виконання) — це спеціальний об'єкт, який містить усю інформацію про поточний запит (які аргументи передав користувач, які поля він хоче побачити тощо).

repository.getAllTrips() — Оскільки для отримання всіх поїздок нам не потрібні ніякі аргументи, ми просто ігноруємо dataFetchingEnvironment і просимо наш репозиторій віддати весь список.

getArgument("id") — А ось тут dataFetchingEnvironment нам вже знадобився! Користувач у запиті написав щось типу getTripById(id: "2"). Цей рядок коду витягує значення "2" з аргументу з назвою "id".

return repository... — Передаємо цей витягнутий tripId у наш репозиторій, він шукає потрібну поїздку і повертає її.

// Кастомний Fetcher 3: Мутація - додавання учасника

```
public DataFetcher<Trip> addParticipantToTripDataFetcher() {
    return dataFetchingEnvironment -> {
        3 String tripId = dataFetchingEnvironment.getArgument("tripId");

        // InputObjectType в GraphQL передається в Java як Map (словник)

        4 Map<String, Object> participantInputMap =
dataFetchingEnvironment.getArgument("newParticipant");

        1-2 // Шукаємо поїздку
        Trip trip = repository.getTripById(tripId);
        if (trip != null) {

            // Створюємо нового учасника з вхідних даних

            3-6 Participant newParticipant = new Participant();
            newParticipant.setId((String) participantInputMap.get("id"));
            newParticipant.setName((String) participantInputMap.get("name"));
            newParticipant.setRole((String) participantInputMap.get("role"));

            1-3 // Додаємо учасника до списку цієї поїздки (змінюємо дані в пам'яті!)
            if (trip.getParticipants() == null) {
                trip.setParticipants(new ArrayList<>());
            }

            4 trip.getParticipants().add(newParticipant);
        }
        return trip; // Повертаємо оновлену поїздку
    };
}
```

// 1 СТАНДАРТНА РЕАЛІЗАЦІЯ DATAFETCHER

// Стандартний PropertyDataFetcher з бібліотеки graphql-java.

// Він просто каже: "візьми значення з поля destination об'єкта Trip".

```
public DataFetcher<String> getTripDestinationStandardFetcher() {
    return PropertyDataFetcher.fetching("destination");
}
}
```

Рядок 3: Витягуємо ID поїздки, в яку треба додати людину.

Рядок 4: Це дуже важливий момент! У GraphQL ми створили InputObjectType (ParticipantInput). Але бібліотека graphql-java автоматично перетворює всі складні вхідні об'єкти на звичайний словник — Map<String, Object>. Тобто participantInputMap — це коробочка, де лежать ключі ("id", "name", "role") і їхні значення ("p3", "Олег", "Водій").

Рядок 1-2: Шукаємо поїздку за ID. Якщо знайшли (вона не null) — працюємо далі.

Рядок 3-6: Створюємо порожній Java-об'єкт Participant і по черзі записуємо в нього дані, витягуючи їх з нашого словника participantInputMap.get(...). (String) попереду потрібен, щоб явно вказати Java, що ми витягуємо саме текст.

Рядок 1-3: Маленька перевірка на безпеку. Якщо список учасників у поїздки ще взагалі не існує (дорівнює null), ми створюємо порожній список (new ArrayList<>), щоб не отримати помилку при додаванні.

Рядок 4: Нарешті додаємо нашого новенького "Олега" до списку. Оскільки trip — це посилання на об'єкт в оперативній пам'яті, зміна зберігається миттєво.

Рядок 6: Повертаємо оновлену поїздку назад у GraphQL, щоб клієнт побачив результат своєї мутації.

Що це: Завдання вимагало використати хоча б одну стандартну реалізацію. Зазвичай ми не пишемо фетчери для кожного дрібного поля (як-от name, date чи budget), бо GraphQL-Java має вбудований PropertyDataFetcher.

Як працює: Цей рядок буквально каже: "Просто знайди в Java-класі метод getDestination() і поверни його значення". У файлі Main ми вручну прив'язали цей фетчер до поля destination типу Trip виключно заради того, щоб виконати вимогу викладача.

DataRepository

```
public class DataRepository {
    // Ось тут у пам'яті будуть зберігатися наші дані (вимога п.5)
    2 private List<Trip> trips = new ArrayList<>();

    public DataRepository() {
4-6     loadData();
    }

    // Метод, який читає JSON і перетворює його на Java-об'єкти
    private void loadData() {
    2     ObjectMapper mapper = new ObjectMapper();
        // Шукаємо файл data.json у папці resources
    1     try (InputStream is = getClass().getResourceAsStream("/data.json")) {
        if (is == null) {
2-4         throw new RuntimeException("Не вдалося знайти файл data.json");
        }
        // Марія Jackson: бере текст і створює з нього готові об'єкти Trip, Participant,
        Preference
    1     trips = mapper.readValue(is, new TypeReference<List<Trip>>() {});
        System.out.println("Дані успішно завантажено! Кількість поїздок: " + trips.size());
    } catch (Exception e) {
4-6     System.err.println("Помилка при читанні файлу даних: " + e.getMessage());
    }
    }

    // Метод для отримання всіх поїздок (знадобиться для GraphQL)
    1 public List<Trip> getAllTrips() {
        return trips;
    }

    // Метод для пошуку конкретної поїздки за ID (знадобиться для GraphQL)
    public Trip getTripById(String id) {
        return trips.stream()
            .filter(trip -> trip.getId().equals(id))
            .findFirst()
            .orElse(null);
    }
}
```

Рядок 2: Створюємо приватну змінну trips. Це звичайний список (масив), який живе в оперативній пам'яті комп'ютера. Саме сюди ми будемо складати всі наші поїздки після того, як прочитаємо їх з файлу.

Рядки 4-6 (Конструктор): Коли в Main ми пишемо new DataRepository(), автоматично викликається цей конструктор. А він, у свою чергу, одразу запускає метод loadData(), щоб "наповнити" наш порожній список даними.

Рядок 2: Створюємо ObjectMapper. Це головний інструмент бібліотеки Jackson, який вміє магичним чином перетворювати текст (JSON) на справжні Java-об'єкти (і навпаки).

Рядок 1: getResourceAsStream("/data.json") шукає ваш файл data.json у папці resources. Конструкція try (...) (так званий try-with-resources) гарантує, що після читання файлу програма автоматично закrije його, щоб не "засмічувати" пам'ять.

Рядок 2-4: Якщо файл не знайдено, викидаємо жорстку помилку і зупиняємо програму (щоб не продовжувати роботу з порожньою базою).

Рядок 1 (Найголовніший!): Метод readValue бере текст із файлу і каже: *"Перетвори цей текст на список об'єктів типу Trip"*. Jackson сам дивиться у ваш Trip.java, бачить там destination, date, participants, шукає такі самі слова у JSON-файлі і розкладає все по полицках. Результат записується в нашу змінну trips.

Рядок 4-6: Якщо JSON написаний з помилкою (наприклад, забули кому), програма не "впаде" повністю, а просто виведе червоне повідомлення про помилку в консоль (блок catch).

Ці два методи — це "ручки", за які смикає наш GraphQLDataFetchers, коли йому треба дістати інформацію.

1 Що робить: Просто повертає весь список поїздок. Використовується для запиту getAllTrips.

2 Що робить: Шукає одну конкретну поїздку за її ідентифікатором. Використовується для запиту getTripById та для нашої мутації. trips.stream() — перетворює список на "потік", щоб його було зручно фільтрувати. .filter(trip -> trip.getId().equals(id)) — залишає в потоці тільки ту поїздку, id якої збігається з тим, що ми шукаємо (наприклад, "2"). .findFirst() — бере першу знайдену поїздку. .orElse(null) — якщо поїздку з таким ID не знайдено (наприклад, шукали ID "999"), метод безпечно поверне null, а не видасть помилку.

"А що буде, якщо в data.json ми додамо нове поле price, якого немає в класі Trip.java?" "Бібліотека Jackson за замовчуванням проігнорує це поле. Вона заповнить лише ті змінні в Java-класі, назви яких збігаються з ключами в JSON. Програма не зламається, але дані про price просто не потраплять в оперативну пам'ять."

Крок 1: Запуск і демонстрація результату (Вау-ефект)

Що робити: Відкривай `Main.java` і натискай зелену кнопку Run.

Що казати: *"Моя програма реалізує GraphQL API для системи планування подорожей. Як бачите в консолі, при старті дані успішно завантажилися з JSON у пам'ять (це пункт 5 завдання). Далі виконуються тестові запити, і результат виводиться як повністю валідний JSON згідно зі специфікацією GraphQL (пункт 8)."*

Крок 2: Схема та ісрархія

Що робити: Відкривай файл `schema.graphqls`.

Що казати: *"Схема зчитується програмою автоматично при старті з файлу `.graphqls` (пункт 1). Тут реалізовано 3 рівні вкладеності (пункт 2): кореневий запит повертає `Trip` (подорож), кожна подорож містить масив `Participant` (учасників), а кожен учасник має об'єкт `Preference` (вподобання)."*

Крок 3: Аргументи та Мутація

Що робити: Залишайся в `schema.graphqls`, виділи мишкою типи `Query` та `Mutation`.

Що казати: *"У схемі є поля з аргументами (пункт 3). Для читання це `getTripById(id)`, а для зміни даних — мутація `addParticipantToTrip`. Для цієї мутації я створила спеціальний аргумент типу `InputObjectType` під назвою `ParticipantInput` (пункт 4)."*

Крок 4: Логіка зміни даних

Що робити: Відкривай консоль внизу (де результати запуску) і показуй другий та третій блоки виводу.

Що казати: *"Ось демонстрація пункту 6. Другим запитом я викликаю мутацію і додаю нового учасника Олега до подорожі в Київ. А третім запитом я роблю звичайний `Query` за цим же ID і показую, що Олег дійсно зберігся в оперативній пам'яті."*

Крок 5: Фетчери (Під капотом)

Що робити: Відкривай клас `GraphQLDataFetchers`.

Що казати: *"Для обробки запитів я написала 3 кастомні `DataFetcher`-и (пункт 7): для отримання всіх подорожей, пошуку за ID та обробки мутації. А також у `Main.java` при налаштуванні `RuntimeWiring` використана одна стандартна реалізація з бібліотеки — `PropertyDataFetcher` для поля `destination`."*

Крок 6: Джерело даних

Що робити: Відкрий `DataRepository` і `data.json`.

Що казати: *"Дані читаються з `data.json` за допомогою бібліотеки `Jackson (ObjectMapper)`. Вона парсить файл і створює колекцію `Java-об'єктів`, з якою потім працюють мої фетчери."*